AI ENGINEERING JOURNEY

WEEK 1	DAY 2	FOUNDATIONS	JUL 7 2026

Embeddings

How LLMs Understand Meaning Through Geometry

The Big Question: Yesterday tokens became integer IDs. Today: how does the model turn those IDs into **meaning**? The answer is embeddings — vectors that encode semantics as geometry. Similar meanings cluster close together in high-dimensional space.

Week 1 — Day Plan

Day	Date	Topic	Status
Day 1	Mon Jul 6	Tokenization	DONE
Day 2	Tue Jul 7	**Embeddings**	YOU ARE HERE
Day 3	Wed Jul 8	Transformer Architecture	Upcoming
Day 4	Thu Jul 9	Training Lifecycle	Upcoming
Day 5	Fri Jul 10	Scaling Laws + Prompt Engineering	Upcoming
Day 6	Sat Jul 11	Structured Output + Architect's Lens	Upcoming
Day 7	Sun Jul 12	Week 1 Project — Model Selection Dashboard	Upcoming

Topics Covered Today

#	Topic	Coverage
1	**What is an Embedding?**	Plain English — vector of floats encoding meaning
2	**Token ID to Vector**	The embedding layer: lookup table connecting Day 1 to Day 2
3	**Geometry of Meaning**	king – man + woman = queen; emergent arithmetic on vectors
4	**Token vs Sentence Embeddings**	What embedding APIs return; how RAG uses them
5	**Similarity Metrics**	Cosine similarity, dot product, L2 — when to use each

6	**FAANG at Scale**	Google Search, Netflix 260M users, Meta FAISS, Amazon product search
7	**Embedding Model Comparison + MTEB**	text-embedding-3-small, MiniLM, Cohere, BGE-M3 — cost, dims, MTEB scores, use case
8	**Architect's Lens**	3 decisions: no model mixing, token limits, cost at scale
9	**Hands-On Exercise**	sentence-transformers — cosine similarity matrix in Python

Section 1 — What is an Embedding? (Plain English)

Yesterday tokens became integers. Today those integers become **meaning.**

An **embedding** is a list of floating-point numbers — a vector — that represents a word, sentence, or document in a high-dimensional space. The model learns to place similar meanings **close together** in that space.

```
"dog" → [0.21, -0.84, 0.53, 0.11, ..., 0.67] (1,536 numbers)
"puppy" → [0.23, -0.81, 0.55, 0.09, ..., 0.65] (1,536 numbers)
"quantum" → [0.91, 0.33, -0.72, 0.88, ..., -0.21] (1,536 numbers)
```

> "dog" and "puppy" are numerically close. "quantum" is far away. **Similarity in meaning = proximity in space.** This single idea powers Google Search, Netflix recommendations, Meta's content ranking, and every RAG system you will build in this course.

Section 2 — From Token IDs to Vectors: The Embedding Layer

Here is the exact path text takes inside a model — connecting Day 1 (Tokenization) to Day 2 (Embeddings):

```
Your text: "Netflix recommends movies"
↓ Tokenizer (Day 1)
Token IDs: [45231, 18, 7823]
↓ Embedding Layer ← TODAY
Vectors: [[0.21, -0.84, ...], [0.11, 0.73, ...], [0.53, -0.22, ...]]
↓ Transformer (Day 3)
Context-aware vectors (each word now understands its neighbours)
↓ Output head
Prediction / generation
```

The **embedding layer** is literally a lookup table: token ID → pre-learned vector. GPT-4 has a table of ~100,000 rows × 12,288 columns. That table has **1.2 billion numbers** in it — just for the first layer of the model.

> Think of the embedding layer as a giant dictionary: you look up a word's ID and get back its vector. The difference from a regular dictionary is that these vectors encode relationship and meaning — not just definitions.

Section 3 — The Famous Equation: king − man + woman ≈ queen

This is not a metaphor. This is actual vector arithmetic that works on real embedding models:

```
vec("king") - vec("man") + vec("woman") ≈ vec("queen")
```

What this tells you: the model learned that the **difference** between "king" and "man" encodes the concept of *royalty without gender*. Adding the woman vector direction gives you "queen."

More real examples from production research:

```
Paris - France + Germany ≈ Berlin (capital city relationship)
Google - Search + Social ≈ Facebook (company + primary function)
Doctor - Hospital + Court ≈ Lawyer (professional + workplace)
walking - walk + swim ≈ swimming (verb tense transfer)
```

> This emergent geometry is **not programmed** — it is *learned* from reading billions of documents. The model discovers that royalty, geography, profession, and grammar all have geometric structure. This is why embeddings are considered one of the most powerful ideas in modern AI.

Section 4 — Token Embeddings vs Sentence Embeddings

This distinction is critical for production system design:

Type	What it Embeds	Output	Used For
Token Embedding	Single token (word piece)	1 vector per token	Inside the model during generation
Sentence Embedding	Full sentence or paragraph	1 vector for the whole input	RAG, search, classification, clustering

When you call an embedding API, you are getting a **sentence embedding** — one vector summarising the entire input. This is what you will use in your RAG system in Weeks 3 and 4.

```
# Sentence embedding — 1 vector for the whole sentence
openai.embeddings.create(
input='How does Netflix recommend movies?',
model='text-embedding-3-small'
)
# Returns: 1 vector of 1,536 floats representing the full sentence meaning
```

Section 5 — Similarity Metrics: How You Measure 'Close'

Three metrics you will encounter in vector databases and search systems:

1. Cosine Similarity (most common)

Measures the angle between two vectors. Range: -1 to 1. Does not care about magnitude — only direction.

```
cosine_similarity(A, B) = (A · B) / (|A| x |B|)

"dog" vs "puppy" → 0.92 (nearly identical direction)
"dog" vs "cat" → 0.74 (related animals)
"dog" vs "finance" → 0.08 (unrelated)
```

Use when: embeddings may have different magnitudes (OpenAI, Cohere). This is the default for most RAG systems.

2. Dot Product

Raw multiplication — cares about both angle AND magnitude.

```
dot_product(A, B) = A · B = sum(Ai x Bi)
```

Use when: embeddings are normalised to unit length (magnitude = 1). Faster to compute than cosine. Used by OpenAI's internal systems and FAISS at Meta.

3. Euclidean Distance (L2)

Straight-line distance between two points in vector space. Lower = more similar.

```
L2(A, B) = sqrt( sum( (Ai - Bi)^2 ) )
```

Use when: magnitude differences carry meaning. Less common for language embeddings. More common for image embeddings.

> **Rule of thumb:** Cosine for language embeddings. Dot product when embeddings are pre-normalised to unit length. Euclidean for image embeddings or when you need true geometric distance.

Section 6 — FAANG at Scale: How Embeddings Work in Production

Google Search — Semantic Search

Before embeddings, Google matched your query to exact keywords. With embeddings, searching "how to fix a slow database" finds results about "database performance optimization" even if those exact words aren't in your query. Google embeds both queries and documents; retrieval is nearest-neighbour search in vector space at **billions of documents per second**.

Netflix — Recommendation at 260M Users

Every movie and show gets embedded based on plot, cast, themes, pacing, genre. Every user's watch history creates a **user embedding** — a vector representing their taste. Recommendations = find movies whose embeddings are closest to the user's embedding. At 260M users x 15,000 titles, this runs as approximate nearest-neighbour search (ANN) using their internal vector search infrastructure.

Meta — FAISS (Facebook AI Similarity Search)

Meta open-sourced FAISS — the most widely used library for billion-scale vector search. It powers their content recommendation, ads targeting, and duplicate detection across billions of posts per day. You will use FAISS in Week 3 when you build your first Vector Search Lab.

Amazon — Product Search

Searching for "comfortable shoes for standing all day" returns orthopedic insoles, supportive sneakers, and gel inserts — products whose *descriptions* embed close to that *query*. No exact keyword match needed. Amazon's product catalogue has 350M+ items, all embedded and searchable semantically.

Section 7 — Embedding Models: What to Use When

Model	Dimensions	Cost	Best Use Case
text-embedding-3-small	1,536	$0.02/1M tokens	Default for English RAG — best cost/quality balance
text-embedding-3-large	3,072	$0.13/1M tokens	High-stakes retrieval where precision matters most
all-MiniLM-L6-v2	384	Free (self-hosted)	On-device, latency-sensitive, no API cost
all-mpnet-base-v2	768	Free (self-hosted)	Higher quality than MiniLM, still free, good default
Cohere embed-v3	1,024	$0.10/1M tokens	Multilingual enterprise — best for 100+ languages
BGE-M3	1,024	Free (self-hosted)	Best open-source multilingual option

Dimension tradeoff: Higher dimensions = more expressiveness = better recall = more storage and slower search. A system with 10M documents at 1,536 dims uses ~59GB RAM. At 384 dims it uses ~15GB. Choose dimensions based on your recall requirements and infrastructure budget.

MTEB — How to Compare Embedding Models Objectively

The **Massive Text Embedding Benchmark (MTEB)** is the industry-standard leaderboard for comparing embedding models. It evaluates models across 56 datasets and 8 tasks: retrieval, clustering, classification, reranking, semantic textual similarity (STS), summarisation, bitext mining, and pair classification.

Model	MTEB Score	Dims	Verdict
text-embedding-3-large	64.6	3,072	Top API model — highest retrieval accuracy
text-embedding-3-small	62.3	1,536	Best cost-efficiency for most production RAG
Cohere embed-v3	64.5	1,024	Best multilingual; competitive with OpenAI large
BGE-M3 (BAAI)	63.6	1,024	Best open-source; free to self-host
all-MiniLM-L6-v2	56.3	384	Fast, lightweight — good for on-device/latency

Architect rule: Always check the MTEB leaderboard (huggingface.co/spaces/mteb/leaderboard) before choosing an embedding model for production. A model that looks good in a demo may rank poorly on retrieval tasks specifically. Filter by the task type that matches your use case.

Section 8 — Architect's Lens: 3 Embedding Decisions in Production

Decision 1 — Never Mix Embedding Models in the Same Index

If you embed your documents with text-embedding-3-small and later switch to text-embedding-3-large, the vectors are incompatible — they live in different geometric spaces. Comparing them gives meaningless similarity scores. Version your embedding model like a database schema. A migration requires re-embedding every document.

```
Rule: embedding_model_version is a schema property of your vector index.
Treat a model change as a breaking migration.
Always store which model generated each embedding alongside the vector.
```

Decision 2 — Input Length Limits Are a Hard Constraint

Embedding models have a maximum input token limit. Exceed it and the model silently truncates your text, losing information from the end. This is a silent failure — no error, just worse results.

```
text-embedding-3-small → max 8,191 tokens
all-MiniLM-L6-v2 → max 256 tokens (very short — fits ~2 paragraphs)
Cohere embed-v3 → max 512 tokens

If your document is 2,000 tokens and your model accepts 256,
you lose 87% of the content silently.
Chunking strategy (Week 3) is designed entirely around this constraint.
```

Decision 3 — Embedding Cost Compounds at Scale

Initial embedding is a one-time cost. But every time you add new documents, update content, or change your model, you pay again. At FAANG scale this dominates budgets.

```
Real example — Google-scale document store:
10 billion documents x avg 500 tokens x $0.02/1M tokens
= $100,000 just to embed the corpus once

With all-MiniLM-L6-v2 (free, self-hosted):
= GPU cost only (~$2,000 one-time on 8xA100 cluster for 24 hours)

This is why Airbnb and Uber run their own embedding servers for high-volume use.
```

Section 9 — Hands-On Exercise: Cosine Similarity in Python

Run this exercise to see embeddings and semantic similarity in action. No API key needed — uses a free local model.

Setup:

```
cd /Users/thaya/AI-Engineering/week-01/notebooks
source /Users/thaya/AI-Engineering/.venv/bin/activate
pip install sentence-transformers # free, no API key needed
```

Create file: week-01/notebooks/day2_embeddings.py

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

sentences = [
'Netflix recommends movies based on your watch history',
'Amazon suggests products you might like',
'Google finds relevant search results for your query',
'The weather is sunny today',
'Machine learning models learn from data',
'Deep learning uses neural networks with many layers',
]

# Generate embeddings — each sentence → 1 vector of 384 floats
embeddings = model.encode(sentences)
print(f'Embedding shape: {embeddings.shape}') # (6, 384)

# Compute cosine similarity between all pairs
def cosine_sim(a, b):
return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print('Similarity matrix (pairs with similarity > 0.5):')
for i in range(len(sentences)):
for j in range(i+1, len(sentences)):
sim = cosine_sim(embeddings[i], embeddings[j])
if sim > 0.5:
print(f' {sim:.2f} | {sentences[i][:40]!r}')
print(f' vs {sentences[j][:40]!r}')
```

What You Will Observe:

- Netflix/Amazon/Google sentences cluster together — all encode the concept of personalised retrieval
- ML and deep learning sentences score high similarity — semantically the same domain
- "The weather is sunny" is isolated — low similarity to all tech sentences
- Similarity is about meaning, not shared words — Netflix and Amazon share no common keywords but embed close
- Try adding your own sentences to build intuition for what 'close' means in vector space

Day 2 — Complete Summary

#	Topic	Key Takeaway	Status
1	**What is an Embedding**	Vector of floats; similar meanings cluster geometrically close in high-dim space	**DONE**
2	**Token ID to Vector**	Embedding layer is a lookup table; GPT-4's has 1.2 billion numbers	**DONE**
3	**Geometry of Meaning**	king – man + woman = queen; emergent arithmetic, not programmed	**DONE**
4	**Token vs Sentence Embeddings**	RAG uses sentence embeddings — 1 vector per chunk, not per token	**DONE**
5	**Similarity Metrics**	Cosine for language; dot product when normalised; L2 for images	**DONE**
6	**FAANG at Scale**	Google Search, Netflix 260M users, Meta FAISS, Amazon 350M products	**DONE**
7	**Embedding Models + MTEB**	text-embedding-3-small default; MiniLM free; Cohere multilingual; MTEB leaderboard for selection	**DONE**
8	**Architect's Lens**	No model mixing; respect token limits; cost compounds at scale	**DONE**
9	**Hands-On Exercise**	sentence-transformers cosine similarity matrix — week-01/notebooks/day2_embeddings.py	**ACTION**

Preview — Day 3: Transformer Architecture (Wednesday, Jul 8)

Now that you know tokens become vectors via the embedding layer, the next question is: **how does the model understand relationships between tokens?** How does it know that "bank" in "river bank" means something different from "bank" in "savings bank"?

The answer is the **Transformer architecture** — specifically the **self-attention mechanism**, the invention that changed everything in 2017 (the 'Attention Is All You Need' paper from Google).

Day 3 will cover:

- The Transformer architecture — encoder, decoder, encoder-decoder variants
- Self-attention: how every token looks at every other token to understand context
- Multi-head attention: why the model needs multiple 'perspectives' simultaneously
- Positional encoding: how the model knows word order without recurrence
- Feed-forward layers, residual connections, layer normalisation
- Architect's Lens: GPT vs BERT vs T5 — which architecture for which task